

Yi Xing Goldsmith — System Developer Documentation

Redacted Version — Internal Use

This is a redacted version of the internal system documentation. All credentials, API keys, admin keys, credential IDs, database base/table identifiers, phone number IDs, server IP addresses, internal domain paths, and login details have been removed or replaced with generic placeholders. The system architecture and operating logic have been preserved for reference.

1. System Overview

The system is a production AI platform for a jewelry retail business, made up of three connected services: a WhatsApp chatbot that handles customer conversations end-to-end, a visual search engine for image-based product matching, and a stock-code lookup service.

1.1 Architecture at a Glance

Component	Technology	Purpose
WhatsApp AI Chat Bot	n8n + WhatsApp Cloud API + LLM	End-to-end WhatsApp customer service automation
Visual Search API	Python FastAPI + CLIP + HNSW	AI image similarity search for jewelry products
Stock Lookup API	Python Flask + Waitress	Stock-code-to-image-URL resolver

1.2 How Everything Connects

- Customer sends a message (text / image / audio) to the business WhatsApp number.
- The messaging platform fires a webhook to the n8n workflow.
- n8n routes the message: audio → transcription; image → visual search; text → intent detection.
- For product enquiries, n8n calls the internal product API to fetch live data.
- For image searches, n8n posts the image to the visual search service, which returns matches.
- The stock-lookup service can resolve product codes to public image URLs for sharing.
- The AI response is generated via an LLM and sent back through the WhatsApp API.
- All conversations and customer records are stored in the database.

2. WhatsApp AI Chat Bot (n8n Workflow)

This is the most complex component — a large n8n workflow (371 nodes) that handles every customer interaction from first contact to response delivery.

Important Note on Messaging Platform

Although one inbound node is legacy-labelled with a different messaging provider's name, the actual messaging platform in production is the WhatsApp Business Cloud API. Phone numbers are received in a "whatsapp":

+<countrycode><number>" format from the webhook payload and stripped to digits before sending replies. Treat the legacy node name as cosmetic only.

2.1 Technology Stack

Service	Platform	Notes
Messaging	WhatsApp Business Cloud API	Configured via a phone number ID in the credentials vault
AI Model (chat)	LLM via OpenRouter	Model: Grok-4-Fast
AI Model (audio)	Whisper (speech-to-text)	Model: whisper-1
CRM Database	Airtable	Main chat/customer base
Escalation CRM	Airtable (separate base)	Staff/escalation contacts
Product API	Internal company API	Requires token authentication
Image Search	Internal Visual Search service	Called over HTTPS with a self-signed certificate in this deployment
Token Auth	Internal token-issuing endpoint	Bearer token required for product API calls, refreshed per request

2.2 Database Tables Referenced

Table	Base	Purpose
Customers	Main CRM base	Customer registry — one record per phone number
Chat logs	Main CRM base	Every conversation turn logged here
Raw incoming messages	Main CRM base	Unprocessed message queue, polled by the scheduler
Staff	Separate escalation base	Staff to notify on escalation

Field-level detail (Chat logs): UserMessage, BotReply, Timestamp, a link to the Customers record, and a detected Intent label.

Field-level detail (Customers): an internal record ID, customer Name, and a From field holding the phone number in international format.

Exact base and table identifiers are stored in the internal configuration reference and are not reproduced in this document.

2.3 Entry Points

The workflow has two entry points that merge into the same processing pipeline:

- Entry Point A — Real-time webhook: receives the live message payload when a customer sends a message, and normalizes it into standardized internal fields (message ID, From, Body, media URL, media count, message type, profile name).
- Entry Point B — Scheduled polling: fires every 20 seconds and checks the staging table for unprocessed records, handling retries and delayed processing.

2.4 Message Type Routing

After entry, a router splits execution into three branches based on message type:

Branch	Condition	Downstream Path
Audio	Message type is audio	Download audio → transcribe → normalize text → customer lookup → intent pipeline
Text	Message type is text	Directly to customer lookup → intent pipeline
Image	Message type is image	Fetch image → download → visual search → image intent pipeline

Audio flow: the audio file is downloaded from the messaging provider's media host using an access token, transcribed via the speech-to-text service, and the transcript is used as the message body (falling back to the raw body if the transcript is empty). Execution then continues identically to the text path.

Image flow: the image's media ID is resolved to a download URL via the messaging provider's Graph API, the image binary is downloaded, and then posted to the internal visual search service, which returns a ranked list of matching products.

2.5 Customer Lookup and New vs. Existing Routing

- The customer table is queried by phone number.
- If no matching record exists, a new customer record is created and the workflow proceeds down a first-contact welcome path.
- If a matching record exists, the message and profile name are set and chat history is loaded.

New Customer Welcome Flow: the first message is checked for live-price intent. If it is a live-price enquiry, the bot fetches the current gold price and replies with buy/sell pricing. Otherwise, it fetches the same pricing context but builds a general welcome message with greeting detection (a full store introduction is used only when the message is a greeting-only first contact).

2.6 Chat History Loading (Existing Customers)

Before intent detection, the last 5 conversation turns are loaded and formatted:

- Read all chat-log records for the customer.
- Extract a usable timestamp from whichever time field is present.
- Keep only records where the user message or bot reply is non-empty.
- Sort oldest to newest, then take the last 5 turns.
- Format each turn as a User/Bot exchange line, joined into a single history string.

If no chat-log records are found (e.g., a first-time lookup race condition), the history string is safely treated as empty, and the AI still responds correctly without prior context.

2.7 Intent Detection

The combined chat history and current message are sent to the LLM for intent classification. The model is instructed to read recent history plus the latest message and return only a JSON object containing an intent label, using a low-confidence fallback label rather than guessing.

The classified intent then routes through a switch with roughly a dozen branches, summarized below:

Intent	Meaning	Handler Summary
--------	---------	-----------------

General FAQ	Generic question about the store, policies, delivery, etc.	Answered directly from FAQ context
Product availability	Asking if a specific product is in stock	Structured extraction → product API lookup → match logic → reply or FAQ fallback
General product	Browsing / general product question	Similar to product availability, with broader search
Product by category	Asking for products in a category	Product API query filtered by category
Order booking	Wants to place an order	Product lookup → order-context reply
Custom order	Wants a bespoke piece	Dedicated prompt → reply → staff notification
Pre-order	Wants to pre-order an upcoming product	Product lookup → pre-order-context reply
Live price	Asking for current gold buy/sell price	Live gold-price lookup → reply
Bulk product/order	Wholesale / bulk enquiry	Dedicated handling → reply → staff notification
Image-link	Wants a direct image/product link	Stock lookup or product API → reply
Blast template / catalog	Internal broadcast operations	Sends an approved template message to a customer list
Fallback	Anything the bot cannot handle confidently	Polite hand-off reply → conversation summary → staff escalation

2.8 Image Message Pipeline

- Extract any caption text accompanying the image.
- Post the downloaded image to the visual search service, requesting a fixed number of top matches.
- Sort and clamp match scores, sanitize image URLs, and pass structured matches downstream.
- Route to a high/medium/low confidence response style based on the top match score.
- Format the matched products into a friendly WhatsApp message and send it.

Score-based routing: scores at or above roughly 0.95 are treated as a confident match; scores at or above roughly 0.85 trigger a moderate-confidence reply offering alternatives; anything lower triggers a low-confidence reply asking the customer to clarify or falls back to FAQ.

Troubleshooting tips: if image search returns nothing, confirm the visual search service is running and its index is built; if it fails outright, verify the network path/certificate configuration to the visual search service hasn't changed; low scores across the board usually indicate a low-quality photo or a stale index.

2.9 Product Availability Flow (Detailed)

- The customer's message and chat history are sent to the LLM with instructions to extract structured product details as JSON only (name, weight, color, gold type, stock code).

- The JSON response is parsed, with graceful fallback to null values if parsing fails.
- A bearer token is issued from the internal token endpoint, then used to call the product API for live product data (stock quantity, price, weight, variations).
- Matching logic first tries a strict match (name/stock code, weight, color, gold type all required); if that yields no results, it falls back to a relaxed match (name/stock code only).
- If exactly one product matches (or exactly one is in stock among matches), the bot replies with that product's details; otherwise it falls through to FAQ or escalation.

Troubleshooting: if the API returns products but the single-match check keeps failing, inspect the structured-extraction node's output; a 404 usually means a wrong stock code or an expired token; check the normalization logic if color/gold-type mismatches are causing zero strict matches.

2.10 Fallback and Staff Escalation Flow

- The bot immediately sends the customer a polite message (language-matched) explaining that a staff member will follow up.
- The chat history is summarized by the LLM into 1–2 sentences covering intent, products mentioned, and specific requests.
- The staff table is queried for all eligible staff members.
- A staff member is selected at random (no priority queue or round-robin tracking in this component) and assigned to the case.
- A WhatsApp template message is built with placeholders for the customer's WhatsApp ID, display name, last message, a local timestamp, and the AI-generated summary, then sent to the assigned staff member.

Staff selection notes: all active staff records are eligible; to exclude someone from receiving escalations, remove their record from the staff table. Troubleshooting: no notification usually means the staff table has no valid records, or the escalation template has expired/been rejected and needs re-approval.

2.11 Response Delivery

All successful AI responses are sent through n8n's native WhatsApp node, with the recipient number derived by stripping the WhatsApp prefix from the original sender field, and the message body taken from the LLM's generated content. After every send, a paired logging step resolves the customer's database record and writes the exchange to the chat-log table — this pattern repeats at every branch endpoint in the workflow.

2.12 Spreadsheet Integration

Several nodes read from and write to an external spreadsheet service, handling order tracking and catalog queries that run in parallel to the main database. Exact spreadsheet identifiers are configured in node credentials and are not reproduced here.

2.13 Credentials Checklist

The following credentials must be configured before the workflow will function (values themselves are stored in the credentials vault, not in this document):

Credential	Used By	What to Configure
Messaging platform API	All send/template nodes	Access token and phone number ID
LLM API (chat)	Intent and reply-generation nodes	Bearer token in the Authorization

		header
Speech-to-text API	Audio transcription node	Bearer token in the Authorization header
Main database API	All main database nodes	Access token scoped to the main base
Escalation database API	Staff lookup node	Access token scoped to the escalation base
Media download API	Image/audio download nodes	Bearer token for the messaging platform's media host
Internal product API	Token-issue and product-lookup nodes	Handled internally via the token-issuing endpoint
Spreadsheet API	Row read/append/update nodes	OAuth2 credential for the spreadsheet service

Note on token hygiene: any long-lived access tokens used for downloading media expire periodically. When they expire, image and audio messages fail silently — the bot still replies, but with empty content. Rotate these regularly and prefer storing them as n8n credential objects rather than embedding them directly in node headers.

2.14 Workflow Troubleshooting Guide

Symptom	Likely Area	Cause & Fix
Customer gets no reply at all	Send-message step	Check execution history; likely an expired access token or malformed recipient number.
Bot replies in the wrong language	Prompt-building step	Language detection follows the customer's latest message; check for mixed-language history confusing the model.
Intent always falls to fallback	Intent classification step	LLM call may be failing (key/rate limit) or returning non-JSON content, which always resolves to fallback.
Image messages error out	Media download step	Access token to the media host has likely expired — refresh it.
Audio not transcribed	Transcription step	Speech-to-text API key invalid or rate-limited — check that credential separately from the chat LLM key.
Product search returns 0 results	Structured extraction / matching step	Inspect the extracted fields; confirm the token-issue step is returning a valid token.
Staff not notified on escalation	Staff lookup / template send	Empty staff table, or the escalation template has expired/needs re-approval.
Gold price shows wrong value	Price lookup step	Requires a valid token; if the price

		API is down, the price fields will be empty.
Chat history missing / bot forgets context	History lookup step	Check the history query is filtering by the correct customer record; confirm logging nodes aren't failing silently.
Webhook fields arrive empty	Webhook configuration	Verify the messaging provider's webhook subscription is correctly configured for message events.
Workflow doesn't trigger at all	Workflow activation	Webhooks only fire when the workflow is toggled Active in n8n.
Scheduled polling not running	Schedule trigger	Confirm the trigger is enabled; an empty queue on each poll is normal and not an error.

3. Visual Search API (FastAPI)

Provides AI-powered image similarity search. Jewelry images are embedded into vectors using a CLIP model and served via an approximate-nearest-neighbor (HNSW) index. When the chat workflow posts a customer photo, this service returns the most visually similar catalog products.

3.1 File Structure

File	Role	Description
app.py	Main application	API endpoints, search logic, preprocessing, optional re-ranking, scheduler
config.py	Configuration	All environment variables, paths, and model settings
img_preprocess.py	Image utilities	Shared resize/normalize helpers used by both indexer and search
indexer script	Index builder	Scans image directories, generates embeddings, builds the index files
requirements.txt	Dependencies	Python packages for this service
systemd service file	Deployment	Linux service definition to run the API as a background process

3.2 Configuration Variables (Summary)

Key settings include: the filesystem locations scanned for product images; where index files are saved; which embedding model is used (changing this requires a full reindex); a minimum similarity score threshold for returned results; an admin key that protects the reindex endpoint (must be changed from any default value in production); HNSW search accuracy/speed tuning; the candidate pool size before re-ranking; base URLs used to construct public image links; and an optional daily-reindex scheduler with a configurable local time.

3.3 Index Files

Three files must always stay in sync: a binary vector index, a NumPy array of embeddings, and a JSON metadata file listing paths/buckets/filenames in the same row order as the embeddings. Row *i* in the embeddings array must correspond exactly to index *i* in the metadata arrays. These files should never be edited manually — always rebuild via the reindex endpoint. If the service crashes mid-reindex, re-running reindex resolves any misalignment. Images that fail to embed are logged to a dedicated "bad images" log file for review.

3.4 API Endpoints (Summary)

Endpoint	Purpose
GET /health	Basic health check, no authentication required
GET /config	Returns current model/threshold/scheduler configuration
POST /reindex	Rebuilds the full index; requires an admin-key header; protected by a self-expiring lock to prevent concurrent runs
POST /search	Accepts an uploaded image and returns ranked product matches with scores and image URLs

A response includes a `low_confidence` flag, set true when the top score is weak or when the gap between the top two results is too small to be a confident, unambiguous match.

3.5 Search Algorithm (Summary)

- Preprocess the query image (orientation correction, RGB conversion).
- Generate multiple crop/variant versions of the query image to improve robustness.
- Embed all variants with the CLIP model.
- Retrieve nearest-neighbor candidates from the index for each variant and merge the candidate sets.
- Score each candidate by its maximum similarity across all query variants.
- Optionally re-rank close-scoring top candidates using classical feature matching for finer discrimination.
- De-duplicate results and apply the confidence check described above.

3.6 Image Preprocessing

A shared resize function keeps the longest image side under a fixed limit, and a shared normalization function applies mild, consistent contrast/brightness/sharpening enhancements. Both functions are applied identically at indexing time and at query time — this consistency is required for accurate similarity scoring, so neither should be changed without a full reindex.

3.7 Indexer

The indexer scans all configured image directories, deduplicates symlinked files, embeds every image in batches, and writes the three output index files. Any image that fails to load or embed is recorded in the bad-images log with its path and error for later review.

3.8 Deployment

Runs as a systemd service on the production Linux server, listening on an internal port, using a Python virtual environment, running as a restricted service user, and configured to restart automatically on failure.

Common operational commands: start / stop / restart / status via `systemctl`, tail logs via `journalctl`, and trigger a manual reindex via an authenticated HTTP request to the reindex endpoint.

3.9 Scheduler

An optional built-in scheduler can trigger a daily automatic reindex at a configured local time; it is disabled by default and, when enabled, internally converts the configured local time to UTC to avoid daylight-saving issues.

4. Stock Lookup API (Flask)

A lightweight service that maps jewelry stock codes to public image URLs by scanning the filesystem and fuzzy-matching incoming queries against an in-memory filename index.

4.1 Overview

Built with Flask and served via a production WSGI server. Runs on an internal port. Can be deployed directly or via a provided Docker container. Uses no external database — the index is a simple in-memory filename map, rebuilt on demand.

4.2 Configuration Variables (Summary)

Key settings include: a public base URL prepended to image paths; an optional admin key protecting the reindex endpoint (leaving it unset makes reindexing publicly accessible, which is not recommended); separate filesystem roots for regular, bulk, and variation product images; and the listening port.

4.3 Auto-Nested Directory Detection

The service automatically detects and prefers a nested subdirectory sharing the parent folder's name if it contains as many or more image files than the parent — this accommodates a common deployment pattern without extra configuration.

4.4 Stock Code Matching Logic

Matching is tried in three tiers, in order: an exact match against the normalized filename stem, a starts-with match, and finally a contains match. Normalization lowercases the input and strips whitespace, underscores, hyphens, dots, and brackets before comparing.

4.5 API Endpoints (Summary)

Endpoint	Purpose
GET /health	Basic health check, no authentication required
GET /debug/dirs	Returns resolved directory paths and configuration for verifying deployment without shell access
POST /reindex	Rebuilds the in-memory filename index for one or all buckets; protected by an admin-key header if configured
GET /stock?code=... / GET /stock/<code>	Looks up an image URL for a given stock code, optionally scoped to a specific bucket

4.6 Docker Deployment

A Dockerfile is provided for containerized deployment using a production WSGI server. When run in Docker, the product image directories must be mounted as read-only volumes, since the container has no access to the host filesystem by default. Environment variables configure the public base URL and admin key at container start.

5. Database Data Model (Summary)

5.1 Customers Table

Phone number (primary lookup key), customer name, first-interaction timestamp, status, and staff notes.

5.2 Chatlogs Table

Linked customer reference, raw incoming message, AI-generated response, detected intent, timestamp, a processed flag used by the polling trigger, and message type (text/image/audio).

5.3 Staff Table

Staff name, WhatsApp number for escalation notifications, role (optionally used for routing), and an active/inactive flag.

6. Credentials, Security & Secrets

CRITICAL: any workflow export file for this system may contain hardcoded API keys and access tokens embedded directly in node headers (for example, media-download tokens and speech-to-text keys). Before sharing any workflow export file or committing it to a repository, rotate every credential below and migrate all of them into n8n credential objects rather than embedding values directly in nodes.

Secret	Typical Location	Rotation Action
Media/platform access token	Media-download node headers	Expires periodically — regenerate via the messaging platform's developer console and update every node that uses it
Speech-to-text API key	Transcription node header	Rotate via the provider's API-key console and update the node header
LLM API key	Chat/intent/generation node headers	Rotate via the LLM provider's console; prefer an n8n credential over a raw header value
Database API key	n8n database credentials	Rotate via the database provider's account settings and update the n8n credential
Admin key (Visual Search)	Visual search environment file	Change from any default value — use a long random string
Admin key (Stock Lookup)	Stock lookup environment/Docker config	Set before deploying — without it, reindex is publicly accessible

7. Setup & Deployment Guide (Summary)

7.1 Server Requirements

Requirement	Minimum / Recommended
OS	A current Debian-based Linux distribution

CPU	4 cores minimum, 8+ recommended for indexing speed
RAM	8 GB minimum, 16 GB recommended (embedding model + vectors held in memory)
Disk	50 GB+ depending on image library size
Python	3.10 or 3.11
Network	Internal access to the visual search and stock lookup service ports
Optional	An NVIDIA GPU significantly speeds up embedding generation

7.2 Visual Search API — Setup Steps (Summary)

- Copy the source files to the target directory and create a Python virtual environment.
- Install dependencies from the requirements file.
- Create and populate the environment configuration file with the required variables (see Section 3.2).
- Create the index output directory.
- Install and enable the provided systemd service.
- Trigger an initial index build via the reindex endpoint (with the admin key).
- Monitor indexing progress via the service logs.

7.3 Stock Lookup API — Setup Steps (Summary)

- Copy the application files to the deployment directory.
- Either install dependencies directly and run the app, or build and run the provided Docker image with the correct volume mounts.
- Verify the health endpoint responds.
- Check resolved directories via the debug endpoint.
- Trigger an initial index build via the reindex endpoint (with the admin key, if configured).

7.4 n8n Workflow

- Import the workflow JSON file into n8n.
- Configure all credentials per the checklist in Section 2.13.
- Replace any hardcoded tokens found in node headers with proper n8n credential objects.
- Verify the visual search service URL configured in the image-search node matches your deployment.
- Point the messaging platform's webhook configuration at your n8n instance.
- Activate the workflow and send a test message to verify the end-to-end flow.

8. Troubleshooting Guide

Symptom	Likely Cause	Resolution
Health check fails	Service not started or crashed on startup	Check service status and recent logs
Search returns "index not ready"	Reindex hasn't been run, or index files are missing	Trigger a reindex; confirm the index directory exists and is writable

Search returns unrelated products	Stale index or a similarity threshold set too low	Trigger a reindex; consider raising the minimum score threshold
All results flagged low confidence	Poor image quality or threshold set too high	Test with a clean, well-lit image; adjust thresholds carefully
Reindex reports already running	A previous reindex crashed and left a stale lock	Check the lock file's age; remove manually if stale
Stock lookup returns not found	Stock code format mismatch or images not in the expected directory	Check the debug/dirs endpoint and compare normalization against the actual filenames
Workflow not triggering	Webhook misconfigured or workflow inactive	Verify webhook configuration and that the workflow is toggled Active
AI responses generic or wrong language	Prompt templates not tailored to your business context	Review and adjust the relevant prompt-building nodes
Database 401 errors in n8n	Expired API key or wrong base identifier	Regenerate the key and verify the base identifier in the affected nodes
AI API 401 errors	Token-issuing step returning a bad/expired token	Check the token endpoint and its credentials
Images missing after reindex	Failed to embed and logged as bad images	Check the bad-images log for corrupted or unreadable files
High memory usage / out of memory	Embedding model plus large vector array held in RAM	Increase available RAM or reduce the set of indexed image directories

9. Security Checklist

Critical items to address before exposing any service to the public internet:

- Change all admin keys from any default value to a long random string, stored only in the environment configuration.
- The image-search response includes a full server-side file path field — consider removing it if the API is ever exposed publicly.
- Internal service ports should not be directly exposed to the internet; use a reverse proxy with HTTPS and firewall rules instead.
- Implement webhook signature validation on the messaging-platform side to prevent spoofed requests.
- Use a dedicated database service account scoped to only the required base and only the permissions actually needed.
- Rotate all API credentials regularly and avoid hardcoding any credential values directly in workflow JSON — use credential objects instead.
- Add rate limiting (e.g., via a reverse proxy or WAF) to any publicly reachable search endpoint.

10. Dependencies Reference

10.1 Visual Search API

Built on FastAPI with an ASGI server, Pillow for image handling, a sentence-transformers CLIP model for embeddings, HNSW for approximate nearest-neighbor search, NumPy for array storage, multipart-upload support,

fast JSON serialization, a progress-bar utility for indexing, a background scheduler for automatic reindexing, and an optional classical computer-vision library for tie-breaking re-ranking.

10.2 Stock Lookup API

Built on Flask with a production WSGI server for direct deployment and an alternative WSGI server option for containerized deployment, plus standard Flask request/response utilities.

11. Ongoing Maintenance

11.1 Adding New Products

- Place new images in the correct directory under the configured image roots.
- Trigger a reindex on the visual search service.
- Trigger a reindex on the stock lookup service.
- Verify with a test search image or stock code lookup.

If the automatic scheduler is enabled, the visual search service will reindex daily at its configured time; the stock lookup service has no built-in scheduler and must be reindexed manually or via a cron job.

11.2 Model Upgrades

The embedding model can be changed via configuration, but doing so invalidates all existing index files since the vector space changes — a full reindex is mandatory afterward, and the new embedding dimension is auto-detected during indexing.

11.3 Monitoring Checklist

Check	Frequency
Visual search service health endpoint	Daily (automated)
Bad-images log	Weekly
Service logs for errors	Weekly
Index freshness (build timestamp in metadata)	After each product batch upload
n8n workflow execution history for failures	Daily
Database storage usage	Monthly
Disk space on the index directory	Monthly

Redaction Notice

The following categories of information have been intentionally removed from this document: all credential and API key values, admin key values, database base/table identifiers, phone number IDs, server IP addresses, internal filesystem paths tied to a specific host, and any account login details. Please refer to the secure internal credentials store and infrastructure documentation for this information.